

FOOmobile

Jogo da Forca – Aplicativo Android

Relatório de Implementação

Versão 1.0 | Data: 18 de maio de 2026

Plataforma: Android (minSdk 26 / targetSdk 36)

Linguagem: Kotlin | UI: Jetpack Compose (Material 3)

Clean Architecture · MVVM · Hilt · Room · Retrofit

1. Sumário Executivo

O **FOOmobile** é um aplicativo Android de jogo da forca desenvolvido em **Kotlin** com **Jetpack Compose**. O projeto segue a arquitetura **Clean Architecture** combinada com o padrão **MVVM**, oferecendo separação clara entre camadas de domínio, dados e apresentação. A injeção de dependências é gerenciada pelo **Hilt (Dagger)**, persistência local via **Room** e consumo de API REST via **Retrofit**.

Informações do Projeto

- **Package:** playfoo.com
- **Versão:** 1.0 (versionCode 1)
- **Kotlin:** 2.0.21 | AGP: 8.10.1 | KSP: 2.0.21-1.0.28
- **Min SDK:** 26 (Android 8.0) | Target SDK: 36 (Android 15)
- **Java:** 11

2. Arquitetura do Projeto

A estrutura de pacotes reflete as três camadas da *Clean Architecture*:

```
playfoo.com/
├── F00mobileApplication.kt    ← Application (Hilt)
├── MainActivity.kt           ← Entry point + NavHost
├── di/
│   └── AppModule.kt         ← Módulo Hilt
├── domain/                   ← Regras de negócio
│   ├── Palavra.kt
│   ├── Jogador.kt
│   ├── Partida.kt
│   ├── JogoDaForca.kt
│   ├── Dificuldade.kt
│   ├── Tema.kt
│   ├── Turma.kt
│   └── AvatarConfig.kt
├── data/                     ← Fontes de dados
│   ├── local/
│   │   ├── AppDatabase.kt
│   │   └── PalavraEntity.kt
│   ├── remote/
│   │   └── ApiService.kt
│   └── repository/
│       └── ForcaRepository.kt
├── viewmodel/
│   └── GameViewModel.kt
└── ui/                       ← Camada de apresentação
    ├── theme/
    ├── menu/
    ├── game/
    ├── perfil/
    ├── dashboard/
    ├── multiplayer/
    └── turma/
```

3. Camada de Domínio

A camada de domínio contém toda a lógica de negócio pura, sem dependências de frameworks Android.

3.1 Modelos de Domínio

Classe	Responsabilidade	Principais atributos
Palavra.kt	Representa uma palavra do jogo	texto: String; contemLetra(), todasLetrasReveladas(), mostraProgresso()
Jogador.kt	Modelo do jogador	id, nome, avatarConfig, turmalId?
Partida.kt	Sessão de jogo em andamento	tema, palavra, jogador, dificuldade, letrasErradas, letrasCorretas, tentativasRestantes
JogoDaForca.kt	Motor do jogo (engine)	iniciarPartida() → Partida
Dificuldade.kt	Enum de dificuldade	FACIL (8 tent. / sem tempo), NORMAL (6 tent. / 120s), DIFICIL (4 tent. / 60s)
Tema.kt	Categoria/tema de palavras	id, nome, palavras: List<Palavra>
Turma.kt	Turma/sala de aula	id, nome, codigo, membros: List<Jogador>
AvatarConfig.kt	Configuração visual do avatar	tonDePele, cabelo, corCabelo, camisa, corCamisa

3.2 Lógica da Partida

A classe Partida centraliza o controle do jogo em andamento e expõe os seguintes métodos:

- tentativa(letra) — registra uma tentativa de letra (correta ou errada)
- venceu() — retorna *true* quando todas as letras foram reveladas
- terminou() — retorna *true* quando o jogo acabou (vitória ou derrota)
- getProgresso() — string com letras reveladas e underscores
- getTentativasRestantes(), getLetrasErradas(), getLetrasCorretas()

4. Camada de Dados

4.1 Banco de Dados Local (Room)

Arquivo	Descrição
AppDatabase.kt	Banco Room v1, entidade: PalavraEntity, exportSchema: false
PalavraEntity.kt	Tabela <i>palavras</i> : id (PK auto), texto, temald, temaNome

Nota: A interface DAO ainda não foi definida; o esquema do banco está criado mas sem consultas implementadas.

4.2 API Remota (Retrofit)

Endpoint	Método	Descrição
/temas	GET	Retorna lista de temas/categorias de palavras
/turmas/{codigo}	GET	Retorna turma pelo código de acesso

4.3 Repositório

A interface `ForcaRepository` define o contrato de acesso a dados com os métodos `getTemas()`, `getTurmas()` e `getTurmaByCodigo(codigo)`. A implementação concreta ainda não foi criada.

5. Injeção de Dependências (Hilt)

O projeto usa **Hilt 2.56.2** configurado com `@HiltAndroidApp` na classe `FOObileApplication`. O único módulo definido até o momento é o `AppModule`:

- Escopo: `@InstallIn(SingletonComponent::class)`
- Provê: instância singleton de `JogoDaForca`

6. Camada de Apresentação (UI)

6.1 Navegação

Gerenciada via NavController e NavHost dentro de MainActivity:

Rota	Tela	Status
menu	MenuScreen	Implementado
jogo	GameScreen	Implementado
multiplayer	MultiplayerScreen	Placeholder
turmas	TurmaScreen	Placeholder
dashboard	DashboardScreen	Placeholder
perfil	PerfilScreen	Placeholder

6.2 Telas Implementadas

MenuScreen

Tela inicial com botões de navegação centralizados: Jogar, Multiplayer, Turmas, Dashboard e Perfil.

GameScreen Completo

Tela principal do jogo da forca. Consome GameViewModel via Hilt e observa GameUiState. Exibe:

- Nome do tema atual
- Avatar do jogador com estados animados (NEUTRO, VITORIA, DERROTA)
- Contador de tentativas restantes (corações)
- Progresso da palavra (letras reveladas + underscores)
- Teclado QWERTY completo com botões coloridos por estado
- Dialog de resultado (vitória/derrota) com opção de jogar novamente

AvatarEditorScreen Completo

Editor visual do avatar do jogador com preview em tempo real. Permite personalizar:

- Tom de pele (6 opções)
- Estilo de cabelo (4 estilos)
- Cor do cabelo (5 cores)
- Tipo de camisa (4 tipos)
- Cor da camisa (6 cores)

Usa `FilterChip` para seleção e botão de salvar para persistir as alterações.

DashboardScreen / MultiplayerScreen / TurmaScreen / PerfilScreen Placeholder

Telas ainda com implementação mínima, exibindo apenas texto de identificação. Estrutura de navegação já preparada para expansão futura.

6.3 Componentes do Jogo (Composables)

Componente	Descrição
AvatarView	Exibe o avatar do jogador (200×200 dp) com base em AvatarConfig e EstadoAvatar. Assets visuais marcados como TODO.
EstadoAvatar	Enum com estados: NEUTRO, ACERTOU, ERROU, VITORIA, DERROTA
BotaoLetra	Botão 40×40 dp para cada letra; cores: cinza (disponível), verde (correta), vermelho (errada)
ProgressoPalavra	Renderiza a palavra com underscores para letras não reveladas
ContadorTentativas	Exibe corações (❤️ restantes / ❤️ perdidos); cor muda de verde → laranja → vermelho
TecladoLetras	Layout QWERTY em 3 linhas (português). Controla estado de cada letra via BotaoLetra

6.4 Componentes de Perfil

Componente	Descrição
SeletorTomDePele	6 círculos de cor com tons brasileiros (#FFDBAC → #4A2912). Indicador visual de seleção.

6.5 Tema Visual

- **Material Design 3** com suporte a *Dynamic Colors* (Android 12+)
- Suporte a modo escuro automático via `isSystemInDarkTheme()`
- Paleta: Purple80/Purple40 + PurpleGrey + Pink (tons complementares)
- Tipografia: 16sp, Normal weight, com line height e letter spacing configurados

7. ViewModel

GameViewModel Implementado

Injetado com `@HiltViewModel`, gerencia o estado reativo do jogo via `StateFlow<GameUiState>`.

GameUiState

Campo	Tipo	Descrição
progresso	String	Palavra com letras reveladas e underscores
tentativasRestantes	Int	Tentativas ainda disponíveis
tentativasMaximas	Int	Total de tentativas da dificuldade
letrasCorretas	Set<Char>	Letras acertadas
letrasErradas	Set<Char>	Letras erradas
estadoAvatar	String	Estado atual do avatar
tema	String	Nome do tema/categoria
dificuldade	Dificuldade	Nível de dificuldade atual
resultado	Resultado	EM_ANDAMENTO / VITORIA / DERROTA

Métodos

- `iniciarPartida()` — inicia nova partida e atualiza o estado
- `tentarLetra(letra)` — processa uma tentativa e atualiza o estado reativo
- `atualizarEstado()` — sincroniza o *GameUiState* com o estado da *Partida*

8. Dependências e Bibliotecas

8.1 AndroidX e Compose

Biblioteca	Versão	Uso
androidx.core:core-ktx	1.10.1	Extensões Kotlin para Android
androidx.activity:activity-compose	1.13.0	Activity com Compose
androidx.lifecycle:lifecycle-runtime-ktx	2.6.1	Lifecycle-aware coroutines
Compose BOM	2026.02.01	Gerenciamento de versões Compose
compose.material3	(via BOM)	Material Design 3
navigation-compose	2.8.9	Navegação entre telas
lifecycle-viewmodel-compose	2.8.7	ViewModel no Compose

8.2 Injeção de Dependências

Biblioteca	Versão
hilt-android	2.56.2
hilt-compiler (KSP)	2.56.2
hilt-navigation-compose	1.2.0

8.3 Persistência Local

Biblioteca	Versão
room-runtime	2.7.1
room-ktx	2.7.1
room-compiler (KSP)	2.7.1

8.4 Rede

Biblioteca	Versão
retrofit	2.11.0
converter-gson	2.11.0
okhttp	4.12.0
logging-interceptor	4.12.0

kotlinx-coroutines-android

1.10.1

8.5 Outros

Biblioteca	Versão	Uso
MPAndroidChart (JitPack)	3.1.0	Gráficos para o Dashboard
junit	4.13.2	Testes unitários
espresso-core	3.7.0	Testes de UI

9. Status Geral da Implementação

Módulo / Feature	Status	Observação
Motor do jogo (lógica de domínio)	Completo	Palavra, Partida, JogoDaForca, Dificuldade totalmente funcionais
Tela de jogo (GameScreen)	Completo	Avatar, teclado, contador, progresso da palavra, dialog de resultado
GameViewModel + GameUiState	Completo	StateFlow reativo, integrado ao Hilt
Editor de Avatar	Completo	Seleção de pele, cabelo, camisa com preview ao vivo
Componentes de UI do jogo	Completo	BotaoLetra, TecladoLetras, ProgressoPalavra, ContadorTentativas, AvatarView
Tema visual (Material 3)	Completo	Dark mode, Dynamic Colors, tipografia configurados
Navegação entre telas	Completo	NavHost com 6 rotas definidas
Injeção de dependências (Hilt)	Completo	AppModule provê JogoDaForca; HiltViewModel no GameViewModel
Esquema do banco de dados (Room)	Parcial	Entidade PalavraEntity criada; DAO e queries ainda não implementados
Integração com API (Retrofit)	Parcial	ApiService definido; repositório sem implementação concreta
Repositório de dados	Parcial	Interface ForcaRepository definida; implementação pendente
Assets do Avatar	Parcial	Estrutura de estados criada (TODO: substituir placeholder por arte final)
Dashboard (estatísticas)	Placeholder	Tela vazia; MPAndroidChart já na dependência pronto para uso
Multiplayer	Placeholder	Tela vazia; modelo Turma/Jogador já na camada de domínio
Perfil do jogador	Placeholder	Tela vazia; modelo Jogador e AvatarConfig já implementados
Turmas / Sala de aula	Placeholder	Tela vazia; modelo Turma já na camada de domínio

10. Próximos Passos Sugeridos

1. Implementar o **DAO do Room** com queries para salvar/buscar palavras localmente
 2. Criar a **implementação concreta do ForcaRepository** integrando Room e Retrofit
 3. Registrar o repositório no **AppModule** via Hilt (binding de interface)
 4. Desenvolver o **Dashboard** com gráficos de desempenho usando MPAndroidChart
 5. Implementar a tela de **Perfil** para exibir dados do jogador e seu avatar
 6. Implementar a tela de **Turmas** (entrar por código, listar membros)
 7. Desenvolver o **Multiplayer** (sala de jogo em tempo real ou por turnos)
 8. Substituir o **placeholder do AvatarView** pelos assets gráficos finais
 9. Adicionar **seleção de tema e dificuldade** antes de iniciar a partida
 10. Implementar o **temporizador** para os modos NORMAL e DIFICIL
-